

- Document 02: Database & Vector Schema
 - CodePerfect Auditor — PostgreSQL + pgvector Technical Specification
 - Overview
 - Migration 001: Core Extensions — 001_extensions.sql
 - Explanation
 - Migration 002: Multi-Tenant Organizations — 002_core_tables.sql
 - Explanation
 - Users & RBAC — 002_core_tables.sql (continued)
 - Explanation
 - Migration 003: Medical Ontology — 003_medical_ontology.sql
 - ICD Code Master Table (with Vector Embedding)
 - SNOMED Concepts Table (with Vector Embedding)
 - SNOMED → ICD-10 Crosswalk Table
 - Explanation
 - Migration 003: CPT/HCPCS Procedure Codes (with HNSW Index)
 - Explanation
 - Migration 005: Revenue Cycle Tables — 005_revenue_cycle.sql
 - Explanation
 - Migration 006: Audit Log & Row Level Security — 006_audit_and_security.sql
 - Audit Log Table (Full Pipeline Traceability)
 - Row Level Security (RLS) — Multi-Tenant Isolation
 - Explanation
 - Migration 018: Per-Organization Settings — 006_audit_and_security.sql
 - Explanation
 - Database Schema Diagram
 - pgvector RPC Functions

Document 02: Database & Vector Schema

CodePerfect Auditor — PostgreSQL + pgvector Technical Specification

Overview

CodePerfect Auditor uses **Supabase (PostgreSQL 15)** as its sole persistence engine. Rather than maintaining a separate external vector database (like Pinecone), we leverage the **pgvector extension** to store 384-dimensional AI embedding arrays in the exact same rows as our relational clinical data. This guarantees ACID-compliant atomic writes across the entire pipeline.

The database is organized into **18 sequential migrations** applied in strict order. Each migration is a standalone **.sql** file in **/migrations/schema/**.

Migration 001: Core Extensions — 001_extensions.sql

The very first query run on a fresh Supabase instance.

```
-- =====
-- Migration 001: Enable required PostgreSQL extensions
-- Run this FIRST before any table creation
-- =====

CREATE EXTENSION IF NOT EXISTS vector;           -- pgvector: enables VECTOR(384)
columns
CREATE EXTENSION IF NOT EXISTS "uuid-oss";      -- enables gen_random_uuid()
```

Explanation

pgvector is the critical extension that transforms PostgreSQL into an AI-capable vector database. Without this, **VECTOR(384)** column types are undefined and all embedding operations fail. **uuid-oss** enables UUID primary keys, which are mandatory for multi-tenant row isolation in Supabase RLS policies.

Migration 002: Multi-Tenant Organizations — 002_core_tables.sql

All clinical data in CodePerfect is scoped to an Organization. A hospital group and an insurance payer each exist as separate Organization rows, and their data never intersects.

```
-- =====
-- Organizations Table
-- Top-level tenant. Could be a hospital, clinic, or RCM vendor.
-- Every piece of data in the system belongs to an organization.
-- =====
CREATE TABLE organizations (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT          NOT NULL,
  slug        TEXT          UNIQUE NOT NULL, -- URL-safe identifier e.g.
"city-general-hospital"
  type        TEXT          NOT NULL
                    CHECK (type IN ('hospital', 'clinic', 'rcm_vendor',
'diagnostic_center', 'insurance_payer')),
  country     TEXT          DEFAULT 'US',
  timezone    TEXT          DEFAULT 'America/New_York',
  is_active   BOOLEAN       DEFAULT TRUE,
  created_at  TIMESTAMPTZ   DEFAULT NOW(),
  updated_at  TIMESTAMPTZ   DEFAULT NOW()
);

-- Trigger: auto-update updated_at on any row change
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_organizations_updated_at
  BEFORE UPDATE ON organizations
  FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```

Explanation

The **slug** column (e.g. "**apollo-chennai**") allows URL-safe routing between hospital portals. The **type** CHECK constraint enforces that only five specific entity types can

exist, preventing orphaned configuration rows. The trigger ensures the `updated_at` timestamp is always accurate without requiring application-layer logic.

Users & RBAC — `002_core_tables.sql` (continued)

```
-- =====
-- Users Table
-- A user belongs to an organization AND optionally a branch.
-- Roles: admin (org-wide), auditor (read-only), coder (submit cases)
-- =====
CREATE TABLE users (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id   UUID                NOT NULL REFERENCES organizations(id) ON DELETE
CASCADE,
  branch_id         UUID                REFERENCES branches(id) ON DELETE SET NULL,
  email             TEXT                NOT NULL UNIQUE,
  full_name         TEXT                NOT NULL,
  role              TEXT                NOT NULL DEFAULT 'coder'
  CHECK (role IN (
    'admin',      -- Full org access: manage users, view all
branches
    'auditor',    -- Read-only across the org
    'coder',      -- Submit coding cases, view own branch results
    'rcm',        -- Revenue Cycle: manage claims, billing,
analytics
    'payer'       -- Insurance adjudicator: /payer/* portal
access only
  )),
  is_active         BOOLEAN            DEFAULT TRUE,
  last_login_at     TIMESTAMPTZ,
  created_at        TIMESTAMPTZ DEFAULT NOW(),
  updated_at        TIMESTAMPTZ DEFAULT NOW()
);

-- Link Supabase Auth (auth.users) to the public users table
ALTER TABLE public.users
  ADD COLUMN IF NOT EXISTS auth_id UUID UNIQUE REFERENCES auth.users(id) ON
DELETE CASCADE;

-- Index for fast auth_id lookups (used on every page load)
CREATE INDEX IF NOT EXISTS idx_users_auth_id ON public.users(auth_id);
```

Explanation

The `role` CHECK constraint enforces RBAC at the database level. The `auth_id` foreign key links each application user to the corresponding `auth.users` row created by Supabase Auth during `supabase.auth.signUp()`. This linkage is how the RLS engine can read the user's `organization_id` from the JWT and enforce row-level isolation.

Migration 003: Medical Ontology — 003_medical_ontology.sql

The three tables that power the AI's medical knowledge base.

ICD Code Master Table (with Vector Embedding)

```
-- =====
-- Migration 002: ICD-10 Master Code Table
-- Primary reference table for all billable codes
-- =====

CREATE TABLE icd_codes (
  code          TEXT          PRIMARY KEY,          -- e.g. "E11.22"
  description    TEXT          NOT NULL,            -- Full clinical
description
  chapter        TEXT,          -- e.g. "Endocrine,
Nutritional, Metabolic"
  category       TEXT,          -- Sub-category grouping
  is_billable    BOOLEAN       DEFAULT TRUE,        -- Only billable codes
are accepted
  is_cc          BOOLEAN       DEFAULT FALSE,        --
Complication/Comorbidity flag
  is_mcc         BOOLEAN       DEFAULT FALSE,        -- Major CC flag (higher
reimbursement)
  version        TEXT          DEFAULT 'ICD-10-CM-2024',
  base_reimbursement NUMERIC   DEFAULT 0,          -- Simulated DRG base
reimbursement ($)
  embedding      VECTOR(384)    -- Semantic embedding
for similarity search
);
```

SNOMED Concepts Table (with Vector Embedding)

```

-- SNOMED Concepts (clinical terminology store)
CREATE TABLE snomed_concepts (
  snomed_code TEXT PRIMARY KEY, -- e.g. "44054006"
  description TEXT NOT NULL, -- SNOMED Fully Specified Name
  (FSN)
  synonyms TEXT[], -- Common synonyms for matching
  semantic_tag TEXT, -- e.g. "(disorder)", "(finding)", "(procedure)"
  hierarchy TEXT, -- e.g. "Clinical finding", "Procedure"
  is_active BOOLEAN DEFAULT TRUE,
  version TEXT DEFAULT 'SNOMED-CT-2024',
  embedding VECTOR(384) -- Semantic embedding for
  fallback similarity search
);

```

SNOMED → ICD-10 Crosswalk Table

```

-- SNOMED → ICD-10 Crosswalk (deterministic mapping backbone)
CREATE TABLE snomed_icd_map (
  id SERIAL PRIMARY KEY,
  snomed_code TEXT NOT NULL REFERENCES snomed_concepts(snomed_code),
  icd_code TEXT NOT NULL REFERENCES icd_codes(code),
  mapping_type TEXT NOT NULL
  CHECK (mapping_type IN ('exact', 'narrower', 'broader',
  'approximate'))),
  -- exact → direct 1:1 semantic match – safe to use
  directly
  -- narrower → ICD is more specific than SNOMED – valid with
  evidence
  -- broader → ICD is less specific – needs specificity
  scoring
  -- approximate → similar but not equivalent – must pass
  validation
  confidence NUMERIC DEFAULT 1.0 CHECK (confidence BETWEEN 0.0 AND 1.0),
  is_primary BOOLEAN DEFAULT TRUE,
  source TEXT DEFAULT 'manual', -- 'manual' | 'umls' | 'snomed-
  official'
  UNIQUE (snomed_code, icd_code)
);

```

Explanation

The `VECTOR(384)` columns in both `icd_codes` and `snomed_concepts` store the mathematical fingerprint of each medical concept. These are generated by running the `SentenceTransformer("all-MiniLM-L6-v2")` model over each code's description

text. The `snomed_icd_map.mapping_type` CHECK constraint is critical — it enforces that the clinical data team must explicitly classify every mapping relationship. An `approximate` mapping will receive additional scrutiny from Node 6's scoring algorithm before being accepted.

Migration 003: CPT/HCPCS Procedure Codes (with HNSW Index)

```
-- CPT/HCPCS Codes Table
CREATE TABLE IF NOT EXISTS public.cpt_hcpcs_codes (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  code        VARCHAR(10)   UNIQUE NOT NULL,      -- 5-digit CPT or alphanumeric
  HCPCS code
  description TEXT          NOT NULL,             -- Official CMS short description
  code_type   VARCHAR(20)   NOT NULL,             -- 'CPT' (Level I) or 'HCPCS'
  (Level II)
  base_price  NUMERIC(10, 2) NOT NULL,             -- National CMS benchmark rate
  (USD)
  embedding   vector(384),      -- SentenceTransformer semantic
  vector
  created_at  TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT
  NULL
);

-- HNSW index for lightning-fast nearest-neighbor vector searches
-- HNSW (Hierarchical Navigable Small World) is significantly faster than IVFFlat
-- at query time and does not require a training step.
CREATE INDEX IF NOT EXISTS cpt_embedding_idx
  ON public.cpt_hcpcs_codes USING hnsw (embedding vector_cosine_ops);
```

Explanation

The **HNSW index** is the core performance optimization for PostgreSQL vector search. Traditional B-tree indexes cannot handle high-dimensional floating point arrays. HNSW builds a multi-layer graph structure over the embedding space, enabling sub-200ms Approximate Nearest Neighbor (ANN) searches across thousands of CPT codes without full table scans. `vector_cosine_ops` specifies that similarity is measured by cosine distance — the standard metric for comparing semantic embeddings.

Migration 005: Revenue Cycle Tables — 005_revenue_cycle.sql

```
-- DRG-based reimbursement reference per ICD code
CREATE TABLE revenue_lookup (
  id SERIAL PRIMARY KEY,
  icd_code TEXT REFERENCES icd_codes(code),
  drg_group TEXT, -- e.g. "DRG-637", "DRG-291"
  base_reimbursement NUMERIC, -- Base Medicare reimbursement ($)
  cc_adjustment NUMERIC DEFAULT 0, -- Added when CC present
  mcc_adjustment NUMERIC DEFAULT 0, -- Added when MCC present
  effective_year TEXT DEFAULT '2024'
);

-- Claims Table (full RCM lifecycle)
CREATE TABLE IF NOT EXISTS public.claims (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID NOT NULL,
  organization_id UUID NOT NULL REFERENCES
public.organizations(id),
  payer_id UUID REFERENCES public.payers(id),
  status TEXT DEFAULT 'DRAFT' NOT NULL
  CHECK (status IN ('DRAFT', 'SUBMITTED', 'ADJUDICATING',
'PAID', 'DENIED', 'PARTIALLY_PAID')),

  -- RCM Financial Metrics
  total_billed_amount NUMERIC(10,2) DEFAULT 0.00, -- Hospital Gross Charge
  total_allowed_amount NUMERIC(10,2) DEFAULT 0.00, -- Payer Approved Amount
  total_paid_amount NUMERIC(10,2) DEFAULT 0.00, -- Actual payer payment
  patient_responsibility NUMERIC(10,2) DEFAULT 0.00, -- Co-pay / Deductible

  -- Data Payload
  claim_data JSONB, -- Snapshot of ICD, CPT, and
financial_summary
  denial_reason TEXT -- Reason text if claim is denied by
payer
);
```

Explanation

The `claims` table is the core of the post-coding Revenue Cycle Management workflow. The four financial columns (`total_billed_amount`, `total_allowed_amount`, `total_paid_amount`, `patient_responsibility`) model the complete financial lifecycle of a single hospital claim. The `claim_data` `JSONB` column stores a denormalized snapshot of the entire coding session (ICD codes, CPT codes, risk

score), ensuring the payer adjudicator sees exactly the same data the coder submitted even if underlying ontology tables are updated later.

Migration 006: Audit Log & Row Level Security —

006_audit_and_security.sql

Audit Log Table (Full Pipeline Traceability)

```
-- =====
-- Migration 006: Audit Log Table
-- Explainability layer – every LangGraph node decision is logged here.
-- =====
CREATE TABLE audit_log (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id        UUID,
  node_name          TEXT                NOT NULL,
  -- node names: doc_processing | clinical_extract | snomed_resolve |
  --              snomed_icd_map | icd_embedding | icd_decision |
  --              audit_comparison | risk_scoring

  input_snapshot     JSONB,              -- Key state fields going INTO this node
  output_snapshot    JSONB,              -- Key state fields coming OUT of this node

  -- LLM call tracking (only set for clinical_extract node)
  model_name          TEXT,              -- e.g. "llama-3.3-70b-versatile"
  prompt_tokens       INTEGER,
  completion_tokens   INTEGER,
  latency_ms          INTEGER,           -- Wall-clock time for this node in ms

  icd_version          TEXT              DEFAULT 'ICD-10-CM-2024',
  snomed_version       TEXT              DEFAULT 'SNOMED-CT-2024',
  status               TEXT              CHECK (status IN ('success', 'fallback_used',
'failed'))
);
```

Row Level Security (RLS) — Multi-Tenant Isolation

```

-- =====
-- Migration 015: Row-Level Security (RLS) Policies
-- Supabase RLS ensures Hospital A can NEVER query Hospital B's data.
-- Even if someone gets a valid JWT, they only see their org's rows.
-- =====

ALTER TABLE clinical_cases  ENABLE ROW LEVEL SECURITY;
ALTER TABLE coding_results  ENABLE ROW LEVEL SECURITY;
ALTER TABLE audit_log        ENABLE ROW LEVEL SECURITY;

-- Helper: Extract org_id from the current user's JWT
-- Supabase stores organization_id in the JWT under app_metadata.
CREATE OR REPLACE FUNCTION current_user_org_id() RETURNS UUID AS $$
BEGIN
    RETURN (
        current_setting('request.jwt.claims', true)::jsonb
        -> 'app_metadata'
        ->> 'organization_id'
    )::UUID;
EXCEPTION WHEN OTHERS THEN
    RETURN NULL;
END;
$$ LANGUAGE plpgsql STABLE SECURITY DEFINER;

-- clinical_cases: users only see their own organization's cases
CREATE POLICY "org_isolation_clinical_cases_select"
ON clinical_cases FOR SELECT
USING (organization_id = current_user_org_id());

CREATE POLICY "org_isolation_clinical_cases_insert"
ON clinical_cases FOR INSERT
WITH CHECK (organization_id = current_user_org_id());

-- coding_results: same organization isolation
CREATE POLICY "org_isolation_coding_results_select"
ON coding_results FOR SELECT
USING (organization_id = current_user_org_id());

-- audit_log: same organization isolation
CREATE POLICY "org_isolation_audit_log_select"
ON audit_log FOR SELECT
USING (organization_id = current_user_org_id());

```

Explanation

RLS is the backbone of multi-tenant security in CodePerfect. The `current_user_org_id()` function reads the `organization_id` directly from the user's JWT at the database kernel level. This means even if a catastrophic bug in the Python FastAPI layer accidentally queries without a `WHERE organization_id = ?` clause,

PostgreSQL physically refuses to return any rows that don't belong to the authenticated user's organization.

The **SECURITY DEFINER** flag ensures the function executes with the privileges of the function owner (a trusted service role), not the calling user — preventing privilege escalation attacks.

Migration 018: Per-Organization Settings —

006_audit_and_security.sql

```
CREATE TABLE IF NOT EXISTS org_settings (  
  organization_id UUID          PRIMARY KEY REFERENCES organizations(id) ON DELETE  
  CASCADE,  
  icd_version      TEXT          NOT NULL DEFAULT 'ICD-11'  
                                CHECK (icd_version IN ('ICD-10', 'ICD-11')),  
  coding_mode      TEXT          NOT NULL DEFAULT 'balanced'  
                                CHECK (coding_mode IN ('aggressive', 'balanced',  
'conservative'))),  
  claim_scheme     TEXT          NOT NULL DEFAULT 'private'  
                                CHECK (claim_scheme IN (  
    'ayushman_bharat', -- PM-JAY govt scheme  
    'cghs',            -- Central Govt Health  
  
    'esi',              -- Employee State Insurance  
    'private'           -- Private insurer / TPA  
  ))  
);
```

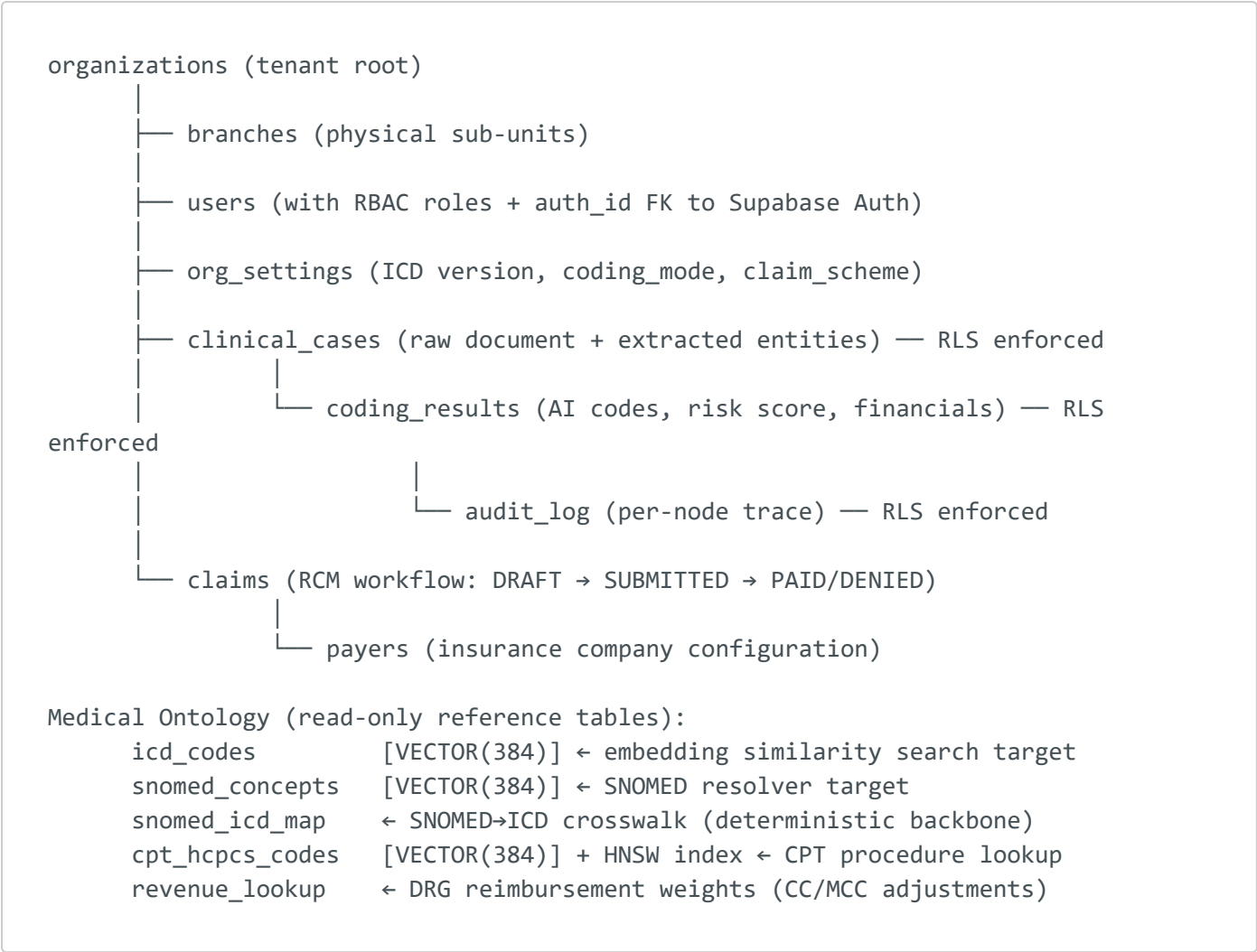
Explanation

org_settings is the per-organization runtime configuration table. It allows the AI pipeline to behave differently for different hospital contexts:

- A government hospital on **ayushman_bharat** scheme is forced into **conservative** coding mode, minimizing audit risk at the expense of revenue capture.
- A private hospital can select **aggressive** mode, maximizing code specificity and DRG weight to capture the highest legally defensible reimbursement.

The AI reads `coding_mode` at pipeline start (from `CodingState.org_id`) and adjusts scoring thresholds accordingly.

Database Schema Diagram



pgvector RPC Functions

The AI nodes query the database using Supabase RPC (Remote Procedure Calls) which execute PostgreSQL functions server-side for maximum performance:

```
-- Function called by Node 5 (icd_embedding_node)
CREATE OR REPLACE FUNCTION match_icd_codes(
  query_embedding VECTOR(384),
  similarity_threshold FLOAT,
  match_count INT
)
RETURNS TABLE (
```

```

code TEXT, description TEXT, chapter TEXT,
is_billable BOOLEAN, is_cc BOOLEAN, is_mcc BOOLEAN,
base_reimbursement NUMERIC, version TEXT, similarity FLOAT
)
LANGUAGE sql STABLE AS $$
    SELECT code, description, chapter, is_billable, is_cc, is_mcc,
           base_reimbursement, version,
           1 - (embedding <=> query_embedding) AS similarity
    FROM icd_codes
    WHERE 1 - (embedding <=> query_embedding) > similarity_threshold
    ORDER BY embedding <=> query_embedding
    LIMIT match_count;
$$;

```

The `<=>` operator is the pgvector cosine distance operator. `1 - cosine_distance` converts distance to similarity (0.0 = completely unrelated, 1.0 = identical). The HNSW index on `icd_codes.embedding` ensures this query executes in under 200ms even with 71,000+ rows.